

Crivo

Banco de dados

Diagrama de relacionamentos, tabelas, formato binário das células, versões e concorrência, integridade e segurança dos dados, consultas prontas e preparação do PostgreSQL.

Copyright (©) 2026 Erik Pimenta

Versão 2.1.1 · Outubro de 2026 · Todos os direitos reservados

Sumário

Visão geral 4

- PostgreSQL é a fonte da verdade 4
- Áreas do modelo 4
- Modo local (sem servidor) 4

Diagrama de relacionamentos 6

- Projetos e versões 6
- Tags, slices e tilesets 7
- Paletas 7

Regras e convenções 8

- Convenções do modelo 8
- Integridade 8
- Exclusões em cascata 9
- Validação antes de gravar 9

Tabelas 11

- project 11
- layer 11
- frame 12
- cell 12
- tag 13
- slice 13
- tileset 14
- revision 14
- palette 15
- palette_color 15
- Tabelas do Django 16

Formato binário das células 17

Versões e concorrência 18

- Um salvamento passo a passo 18
- Restaurar uma versão 18

Tamanho e retenção 18

Integridade e segurança dos dados 19

Do pedido até a transação 19

Revisões como rede de segurança 19

Leituras que não alteram nada 19

Consultas prontas 20

Projetos mais recentes 20

Espaço ocupado por projeto 20

Taxa de compressão das células 20

Cores de uma paleta 20

Versões de um projeto 21

Tags e modos de mistura 21

Projetos com miniatura suspeita 21

Como preparar o banco 22

Com Docker (recomendado) 22

PostgreSQL próprio 22

Backup e restauração 22

Desempenho 22

Evolução do esquema 23

Migração para nomes em inglês 23

Migração dos recursos do Aseprite 24

Como mudar o esquema 25

Capítulo 1

Visão geral

PostgreSQL é a fonte da verdade

Quando o Crivo roda com o servidor Django, tudo o que o editor salva vai para o PostgreSQL 17: projetos, camadas, quadros, o conteúdo de cada célula, as versões anteriores e as paletas. O editor no navegador nunca acessa o banco direto; ele fala só com a API, que valida cada projeto antes de gravar.

Sem variáveis de banco, o Django usa SQLite num arquivo local. O esquema é o mesmo, criado pelas mesmas migrações; só mudam os tipos físicos (por exemplo, `uuid` vira `char(32)` e `bytea` vira `BLOB`). SQLite é ótimo para uso pessoal; para várias pessoas ao mesmo tempo, use PostgreSQL.

> **Versão 2.1.0:** a migração `0004_aseprite_features` acrescenta os recursos do Aseprite: modo de cor no projeto, modo de mistura, grupos, referência e tiles nas camadas, células vinculadas e de tiles, e as tabelas `tag`, `slice` e `tileset`. Ela só acrescenta colunas (com valor padrão ou nulas) e tabelas, então nada é reescrito; rode `python manage.py migrate` (capítulo 10).

> **Versão 2.0.0:** as tabelas, as colunas, as restrições e os índices passaram a ter nomes em inglês (`projeto` virou `project`, `cor_paleta` virou `palette_color` e assim por diante). A migração `0003_english_schema` só renomeia, sem perder dados: basta rodar `python manage.py migrate` (capítulo 10).

Áreas do modelo

Área	Tabelas	Guarda
Projetos	`project`, `layer`, `frame`, `cell`	O desenho: tamanho, modo de cor, ordem e hierarquia das camadas, quadros, e os pixels de cada combinação quadro × camada
Animação e tiles NOVO	`tag`, `slice`, `tileset`	Tags da animação, slices (áreas nomeadas) e conjuntos de tiles das camadas de tiles
Versões	`revision`	Cópias comprimidas das versões anteriores de cada projeto
Paletas	`palette`, `palette_color`	Paletas predefinidas e as criadas pelas pessoas
Django	`django_migrations`, `django_content_type`	Controle das migrações e tipos de conteúdo

Modo local (sem servidor)

Sem API, o editor grava no próprio navegador, num esquema mais simples:

Onde	Chave	Conteúdo
IndexedDB `crivo`, loja `projects`	`id` (texto `local-...`)	`{id, name, data, thumbnail, revision, updated_at}`, em que `data` é o projeto no formato `.crivo`

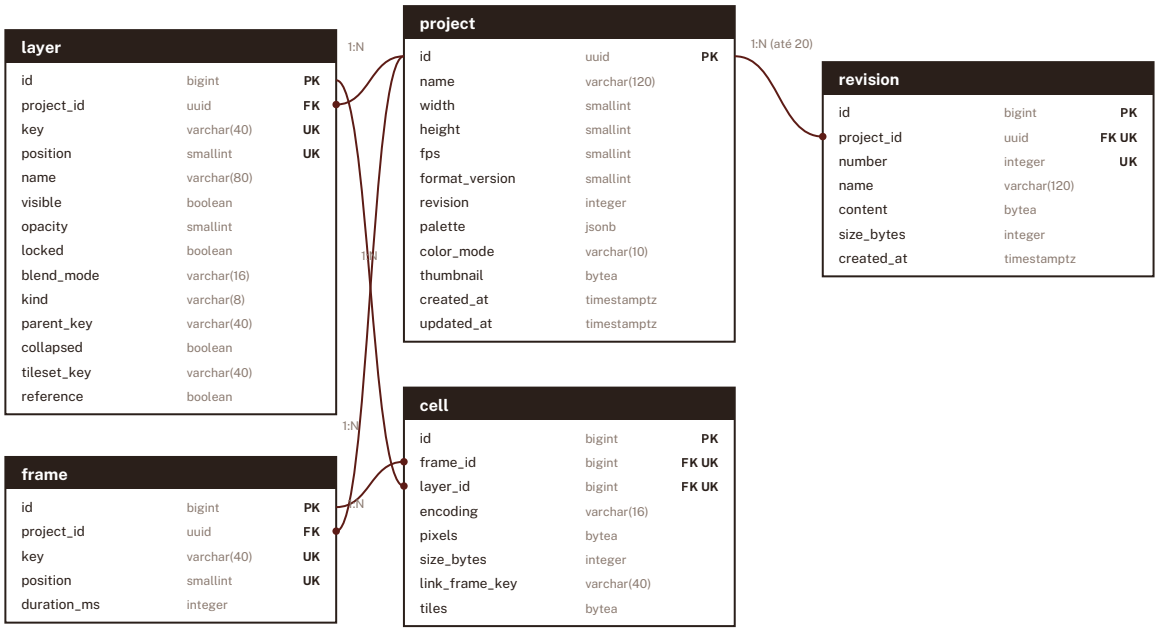
Onde	Chave	Conteúdo
IndexedDB `crivo`, loja `draft`	`current`	O último estado do editor, para recuperar a sessão
`localStorage`	`crivo.palettes`	Lista das paletas criadas no navegador
`localStorage`	`crivo.*`	Preferências (ferramenta, pincel, tema, cores recentes e opções das ferramentas), conferidas ao abrir; valor inválido volta ao padrão

Na 2.0.0, as lojas `projetos` e `rascunho` da 1.x viram `projects` e `draft`: na primeira abertura, o editor converte cada registro e só depois apaga as antigas, sem perder nenhum projeto.

Capítulo 2

Diagrama de relacionamentos

Projetos e versões

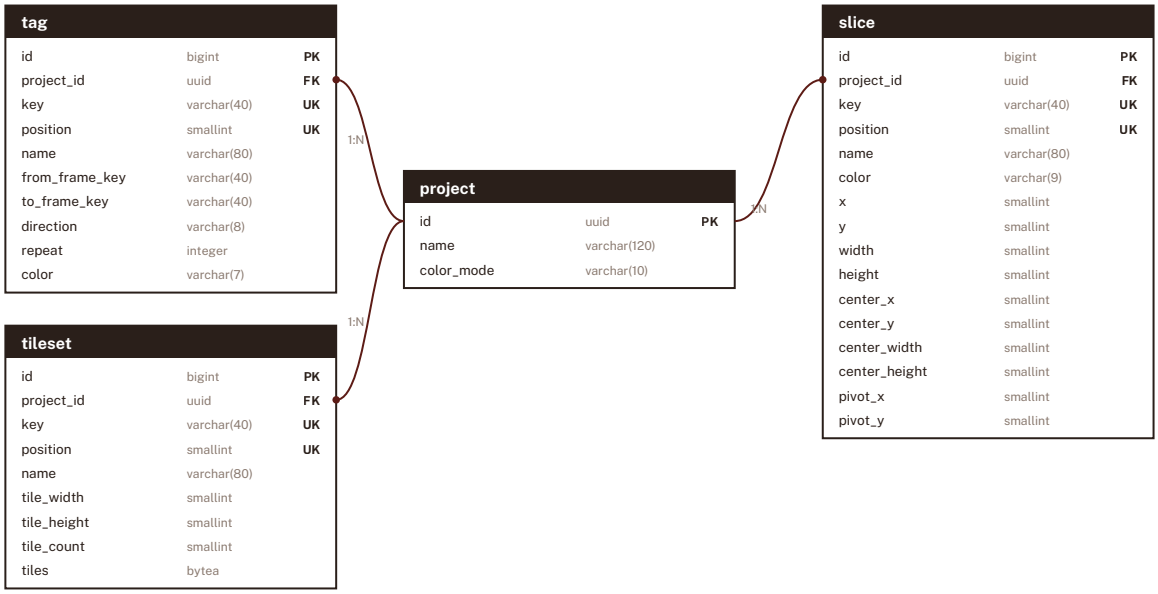


project 1:N layer · project 1:N frame · frame 1:N cell · layer 1:N cell · project 1:N revision

Uma célula é a interseção de um quadro com uma camada. Num projeto com 3 camadas e 4 quadros há até 12 células, mas só as que têm algum pixel pintado são gravadas: uma célula ausente significa "tudo transparente".

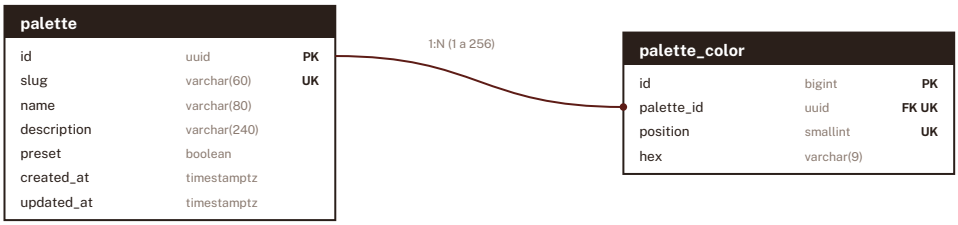
Na 2.1.0, `layer.parent_key` aponta para a `key` de uma camada do tipo grupo do mesmo projeto, `layer.tileset_key` para a `key` de um `tileset`, e `cell.link_frame_key` para a `key` do quadro cuja célula (da mesma camada) esta compartilha. São ligações lógicas pela chave do editor, conferidas pela API antes de gravar, e não chaves estrangeiras: assim a gravação inteira continua sendo uma troca de linhas dentro de uma transação.

Tags, slices e tilesets



project 1:N tag · project 1:N slice · project 1:N tileset (todas com a posição e a chave do editor)

Paletas



palette 1:N palette_color, com a posição de cada cor

As paletas não se ligam aos projetos. Cada projeto guarda a sua própria lista de cores em `project.palette` (JSON), para que apagar ou mudar uma paleta nunca altere um desenho salvo.

Capítulo 3

Regras e convenções

Convenções do modelo

- Nomes de tabelas e colunas em inglês, no singular e em minúsculas (`project`, `palette_color`). Até a 1.2.0 eles eram em português; a migração `0003_english_schema` os renomeou (capítulo 10).
- `project` e `palette` usam `uuid` como chave primária: os endereços da API não revelam quantos projetos existem e podem ser gerados sem consultar o banco.
- As demais tabelas usam `bigint` gerado por identidade (`GENERATED BY DEFAULT AS IDENTITY`).
- `key` em `layer`, `frame`, `tag`, `slice` e `tileset` é o identificador criado pelo editor (por exemplo, `cmvihr...`). É única dentro do projeto e é o que liga as células no arquivo `.crivo`.
- `position` começa em 0. Nas camadas, 0 é a camada de baixo; nos quadros, 0 é o primeiro.
- Datas são `timestamptz`, em UTC no banco. O Django converte para America/Sao_Paulo (ou o fuso de `CRIVO_TIMEZONE`).

Integridade

Restrição	Tabela	Regra
`project_width_valid`, `project_height_valid`	`project`	Largura e altura entre 1 e 1024
`project_fps_valid`	`project`	FPS entre 1 e 60
`layer_key_unique`, `layer_position_unique`	`layer`	Chave e posição únicas no projeto
`layer_opacity_valid`	`layer`	Opacidade entre 0 e 100
`frame_key_unique`, `frame_position_unique`	`frame`	Chave e posição únicas no projeto
`cell_unique`	`cell`	No máximo uma célula por quadro e camada
`project_color_mode_valid` NOVO	`project`	`rgba`, `indexed` ou `grayscale`
`layer_blend_mode_valid` NOVO	`layer`	Um dos 19 modos de mistura (capítulo 4)
`layer_kind_valid` NOVO	`layer`	`image`, `group` ou `tilemap`
`layer_parent_not_self` NOVO	`layer`	Uma camada não pode ser o próprio grupo
`layer_tileset_only_tilemap` NOVO	`layer`	`tileset_key` só em camadas `tilemap`
`tag_key_unique`, `tag_position_unique` NOVO	`tag`	Chave e posição únicas no projeto
`tag_direction_valid`, `tag_repeat_valid`, `tag_color_valid` NOVO	`tag`	`forward`, `reverse` ou `pingpong`; repetições até 65535; cor `#rrggbb`

Restrição	Tabela	Regra
`slice_key_unique`, `slice_position_unique` NOVO	`slice`	Chave e posição únicas no projeto
`slice_origin_valid`, `slice_width_valid`, `slice_height_valid`, `slice_color_valid` NOVO	`slice`	Origem entre −1024 e 1024, largura e altura entre 1 e 1024, cor `#rrggbb` ou `#rrggbbaa`
`slice_center_complete`, `slice_pivot_complete` NOVO	`slice`	O centro de 9-slice tem os quatro campos ou nenhum; o pivô, os dois ou nenhum
`tileset_key_unique`, `tileset_position_unique` NOVO	`tileset`	Chave e posição únicas no projeto
`tileset_tile_width_valid`, `tileset_tile_height_valid`, `tileset_tile_count_valid` NOVO	`tileset`	Tile de 1 a 256 pixels de lado; de 1 a 4096 tiles
`revision_number_unique`	`revision`	Número de versão único no projeto
`palette_color_position_unique`	`palette_color`	Uma cor por posição da paleta
`palette_color_hex_valid`	`palette_color`	Texto `#rrggbb` ou `#rrggbbaa` em minúsculas
`slug` único	`palette`	Identificador estável das paletas predefinidas (as criadas pelas pessoas têm `slug` nulo)

Exclusões em cascata

As chaves estrangeiras são criadas pelo Django como `DEFERRABLE INITIALLY DEFERRED`, e a exclusão em cascata é feita pelo próprio Django dentro da transação: excluir um projeto apaga as camadas, os quadros, as células, as tags, os slices, os tilesets e as versões; excluir uma paleta apaga as cores. Se você apagar linhas direto pelo SQL, apague primeiro as filhas (`cell`, depois `layer` e `frame`, `revision`) ou use `DELETE ... USING` na ordem certa.

Validação antes de gravar

A API não confia no navegador. Antes de qualquer gravação, `editor/services.py` confere:

- o campo `format` igual a `crivo` e `version` até 2 (um documento da versão 1, com as chaves em português, é convertido antes por `upgrade_document`), tamanho entre 1 e 1024, FPS entre 1 e 60, de 1 a 64 camadas e de 1 a 512 quadros;
- identificadores de camadas e quadros presentes, com até 40 caracteres e sem repetição entre si;
- toda célula apontando para um quadro e uma camada que existem, sem duplicatas;
- o RLE de cada célula decodificando para exatamente largura × altura pixels;
- cores da paleta em hexadecimal válido e miniatura em PNG de até 512 KB, conferida pela assinatura do arquivo;
- na 2.1.0 NOVO, as chaves opcionais: modo de cor e modo de mistura conhecidos, grupo que existe, é do tipo grupo e não forma ciclo, tileset só em camadas de tiles, vínculo para uma célula da mesma camada que não é ela mesma nem outro vínculo, tiles dentro do tileset, tags com quadros que existem, slices e tilesets dentro dos limites (até 256 tags, 256 slices e 32 tilesets por projeto, tiles de até 256 × 256 e até 4096 por tileset). Erros respondem 422, com a mensagem em português.

Os limites de pedido (tamanho do corpo, aninhamento do JSON e escritas por minuto) são aplicados antes, na camada HTTP, e estão no capítulo 7.

Capítulo 4

Tabelas

Tipos conforme o PostgreSQL. Na coluna Nulo, "sim" indica que o campo aceita `NULL`.

project

Um desenho com suas dimensões e metadados. Os pixels ficam em `cell`.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária, gerada pela aplicação
`name`	`varchar(120)`	não	Nome mostrado no editor e na lista
`width`	`smallint`	não	Colunas da matriz (1 a 1024)
`height`	`smallint`	não	Linhas da matriz (1 a 1024)
`fps`	`smallint`	não	Velocidade da animação (1 a 60)
`format_version`	`smallint`	não	Versão do formato `.crivo` usada (hoje 2)
`revision`	`integer`	não	Contador de salvamentos, usado no controle de concorrência. Começa em 1
`palette`	`jsonb`	não	Lista de cores do projeto em hexadecimal
`color_mode` NOVO	`varchar(10)`	não	Modo de cor: `rgba` (padrão), `indexed` ou `grayscale`
`thumbnail`	`bytea`	sim	PNG de até 128 px gerado pelo editor
`created_at`	`timestamptz`	não	Criação
`updated_at`	`timestamptz`	não	Último salvamento (índice decrescente `project_updated_idx`)

layer

As camadas do projeto, na ordem de empilhamento. Valem para todos os quadros.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`project_id`	`uuid`	não	Projeto (FK, indexada)
`key`	`varchar(40)`	não	Identificador da camada no editor; único no projeto
`position`	`smallint`	não	Ordem, 0 = camada de baixo; única no projeto
`name`	`varchar(80)`	não	Nome da camada

Campo	Tipo	Nulo	Descrição
`visible`	`boolean`	não	Entra ou não na composição e nas exportações
`opacity`	`smallint`	não	0 a 100
`locked`	`boolean`	não	O editor impede desenhar nela
`blend_mode` NOVO	`varchar(16)`	não	Modo de mistura (padrão `normal`): `normal`, `multiply`, `screen`, `overlay`, `darken`, `lighten`, `color-dodge`, `color-burn`, `hard-light`, `soft-light`, `difference`, `exclusion`, `hue`, `saturation`, `color`, `luminosity`, `addition`, `subtract` ou `divide`
`kind` NOVO	`varchar(8)`	não	`image` (padrão), `group` ou `tilemap`
`parent_key` NOVO	`varchar(40)`	sim	`key` do grupo que contém a camada; nulo no nível de cima
`collapsed` NOVO	`boolean`	não	Grupo fechado na lista de camadas
`tileset_key` NOVO	`varchar(40)`	sim	`key` do tileset usado; só em camadas `tilemap`
`reference` NOVO	`boolean`	não	Camada de referência: aparece no editor e fica fora das exportações

frame

Os quadros da animação, em ordem.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`project_id`	`uuid`	não	Projeto (FK, indexada)
`key`	`varchar(40)`	não	Identificador do quadro no editor; único no projeto
`position`	`smallint`	não	Ordem, 0 = primeiro quadro; única no projeto
`duration_ms`	`integer`	sim	Duração própria do quadro. Nulo = 1000 / fps

cell

Os pixels de um quadro numa camada, em RLE.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`frame_id`	`bigint`	não	Quadro (FK, indexada)
`layer_id`	`bigint`	não	Camada (FK, indexada)

Campo	Tipo	Nulo	Descrição
`encoding`	`varchar(16)`	não	Codificação de `pixels`. Hoje sempre `rle-rgba`
`pixels`	`bytea`	não	Pares (corrida em varint, cor RGBA), ver o capítulo seguinte
`size_bytes`	`integer`	não	Tamanho de `pixels`, para relatórios sem ler o conteúdo
`link_frame_key` NOVO	`varchar(40)`	sim	Célula vinculada: `key` do quadro cuja célula (da mesma camada) esta compartilha. `pixels` continua com uma cópia
`tiles` NOVO	`bytea`	sim	Camadas de tiles: RLE dos índices de tiles (colunas × linhas, inteiros de 32 bits). `pixels` continua com os pixels já desenhados

tag

As tags da animação, na ordem em que aparecem.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`project_id`	`uuid`	não	Projeto (FK, indexada)
`key`	`varchar(40)`	não	Identificador da tag no editor; único no projeto
`position`	`smallint`	não	Ordem, a partir de 0; única no projeto
`name`	`varchar(80)`	não	Nome da tag
`from_frame_key`	`varchar(40)`	não	`key` do primeiro quadro
`to_frame_key`	`varchar(40)`	não	`key` do último quadro
`direction`	`varchar(8)`	não	`forward` (padrão), `reverse` ou `pingpong`
`repeat`	`integer`	não	Repetições, 0 = sem fim (até 65535)
`color`	`varchar(7)`	não	Cor da faixa, `#rrggbb`

slice

Áreas nomeadas sobre a matriz, com centro de 9-slice e pivô opcionais.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`project_id`	`uuid`	não	Projeto (FK, indexada)
`key`	`varchar(40)`	não	Identificador do slice no editor; único no projeto

Campo	Tipo	Nulo	Descrição
`position`	`smallint`	não	Ordem, a partir de 0; única no projeto
`name`	`varchar(80)`	não	Nome do slice
`color`	`varchar(9)`	não	Cor do contorno, `#rrggbb` ou `#rrggbbaa` (padrão `#0000ff`)
`x`, `y`	`smallint`	não	Canto superior esquerdo (−1024 a 1024)
`width`, `height`	`smallint`	não	Tamanho (1 a 1024)
`center_x`, `center_y`, `center_width`, `center_height`	`smallint`	sim	Centro de 9-slice, relativo ao slice; os quatro ou nenhum
`pivot_x`, `pivot_y`	`smallint`	sim	Pivô, relativo ao slice; os dois ou nenhum

tileset

Os conjuntos de tiles usados pelas camadas de tiles.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`project_id`	`uuid`	não	Projeto (FK, indexada)
`key`	`varchar(40)`	não	Identificador do tileset no editor; único no projeto (é o que `layer.tileset_key` guarda)
`position`	`smallint`	não	Ordem, a partir de 0; única no projeto
`name`	`varchar(80)`	não	Nome do tileset
`tile_width`, `tile_height`	`smallint`	não	Tamanho de cada tile (1 a 256)
`tile_count`	`smallint`	não	Quantidade de tiles (1 a 4096), contando o tile 0, que é o vazio
`tiles`	`bytea`	não	Todos os tiles em RLE (o mesmo formato de `cell.pixels`), um depois do outro, na ordem

revision

Versões anteriores de um projeto. Cada salvamento guarda aqui o estado que estava no banco antes dele.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`project_id`	`uuid`	não	Projeto (FK, indexada)

Campo	Tipo	Nulo	Descrição
`number`	`integer`	não	Valor de `project.revision` naquele momento; único no projeto
`name`	`varchar(120)`	não	Nome do projeto naquela versão
`content`	`bytea`	não	O projeto inteiro no formato `.crivo`, em JSON comprimido com zlib. Versões guardadas antes da 2.0.0 continuam no formato 1 e são convertidas ao restaurar
`size_bytes`	`integer`	não	Tamanho de `content`
`created_at`	`timestamptz`	não	Quando a versão foi guardada

palette

Paletas predefinidas (criadas pelas migrações) e paletas das pessoas.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`slug`	`varchar(60)`	sim	Identificador estável das predefinidas (`game-boy`, `nes`…); único
`name`	`varchar(80)`	não	Nome mostrado
`description`	`varchar(240)`	não	Texto curto, vazio nas paletas das pessoas
`preset`	`boolean`	não	A API recusa alterar ou excluir quando verdadeiro
`created_at`	`timestamptz`	não	Criação
`updated_at`	`timestamptz`	não	Última alteração

palette_color

As cores de cada paleta, na ordem.

Campo	Tipo	Nulo	Descrição
`id`	`bigint`	não	Chave primária
`palette_id`	`uuid`	não	Paleta (FK, indexada)
`position`	`smallint`	não	Ordem da cor, a partir de 0; única na paleta
`hex`	`varchar(9)`	não	`#rrggbb` ou `#rrggbbaa`, minúsculo

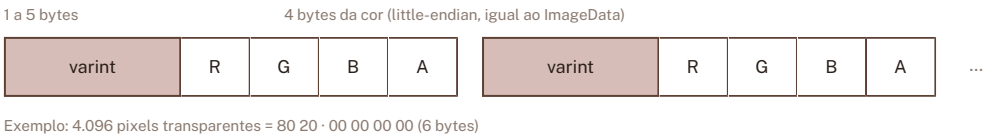
Tabelas do Django

Tabela	Para que serve
`django_migrations`	Registra quais migrações já rodaram (`editor.0001_inicial` , `editor.0002_paletas_predefinidas` , `editor.0003_english_schema` , `editor.0004_aseprite_features` e as do Django)
`django_content_type`	Catálogo interno de modelos do Django

Capítulo 5

Formato binário das células

A coluna `cell.pixels` usa o mesmo formato que o editor (JavaScript), o servidor (Python) e a ferramenta em Java leem e escrevem. A matriz é percorrida linha por linha, da esquerda para a direita, e cada sequência de pixels iguais vira um par:



Exemplo: 4.096 pixels transparentes = 80 20 · 00 00 00 00 (6 bytes)

Parte	Tamanho	Conteúdo
Corrida	1 a 5 bytes	Quantos pixels seguidos têm a cor. Varint: 7 bits por byte, do menos para o mais significativo; o bit 8 ligado indica que há outro byte
Cor	4 bytes	R, G, B e A, nessa ordem. A = 0 é transparente

A soma das corridas é sempre largura × altura; a API recusa células que não fecham. No arquivo `.crivo` e na API, os mesmos bytes viajam em base64 no campo `rle` (dentro de `cells`). Para ler os pixels direto no banco, há três opções:

- pela API, em `GET /api/v2/projects/{id}/export`, que já decodifica, compõe as camadas visíveis e devolve a matriz em JSON, CSV, JavaScript, Python, C ou Java;
- pela API, em `GET /api/v2/projects/{id}/png?frame=N&scale=E`, que lê as células do quadro, compõe as camadas visíveis com a mesma fórmula do editor e devolve um PNG ampliado por vizinho mais próximo (lado final de até 2048 px), gerado em Python puro, só com `zlib` e CRC, sem Pillow;
- pelo código, com `editor.rle.decode(data, width * height)`, que devolve um `array('I')` com um inteiro por pixel (`R | G << 8 | B << 16 | A << 24`).

Capítulo 6

Versões e concorrência

Um salvamento passo a passo

1. O editor envia `PUT /api/v2/projects/{id}` com o projeto em `data` e o número de `revision` que ele conhece.
2. Numa transação, a API trava a linha do projeto com `SELECT ... FOR UPDATE`. Outro salvamento do mesmo projeto espera esse terminar.
3. Se a revisão enviada for diferente de `project.revision`, a transação é desfeita e a resposta é 409, com a revisão atual em `current_revision`. Nada é gravado.
4. O estado atual do banco vira uma linha em `revision` (JSON do `.crivo` comprimido com zlib). Se houver mais de `CRIVO_MAX_REVISIONS` versões, as mais antigas são apagadas.
5. As camadas, os quadros, as tags, os slices e os tilesets antigos são removidos (com as células) e os novos são inseridos com `bulk_create`, em lotes.
6. `project.revision` aumenta 1, e a resposta devolve o novo número, que o editor guarda para o próximo salvamento.

> Como a troca acontece dentro de uma única transação, quem lê o projeto ao mesmo tempo vê a versão antiga inteira ou a nova inteira, nunca uma mistura.

Restaurar uma versão

`POST /api/v2/projects/{id}/revisions/{number}/restore` descomprime a versão `number` e faz um salvamento normal com ela. Por isso a versão que estava em uso também vai para o histórico, e restaurar pode ser desfeito restaurando de novo. Versões guardadas antes da 2.0.0 estão no formato 1, com as chaves em português: ao restaurar, o servidor as converte para o formato 2 (`editor.services.upgrade_document`) antes de validar e gravar.

Tamanho e retenção

Cada versão guarda o projeto inteiro, mas comprimido duas vezes (RLE dentro do JSON e zlib por fora). Um sprite 32×32 com 4 quadros costuma ocupar menos de 2 KB por versão. Para guardar mais ou menos versões, ajuste `CRIVO_MAX_REVISIONS`; o corte acontece no próximo salvamento de cada projeto.

Capítulo 7

Integridade e segurança dos dados

Desde a versão 1.1.0 o servidor tem uma camada de proteção na frente do banco. Nenhuma delas muda o esquema; todas existem para que só dados válidos, de tamanho razoável e enviados de forma legítima cheguem a uma transação.

Do pedido até a transação

Etapa	O que é conferido	Se falhar
Tamanho do corpo	Até `CRIVO_MAX_UPLOAD_MB` (padrão 24 MB)	413
CSRF ou token	Cookie e cabeçalho CSRF do editor, ou `Authorization: Bearer` comparado em tempo constante com `CRIVO_API_TOKEN`	403
Limite de escritas	`POST`, `PUT` e `DELETE` por IP, numa janela de um minuto (`CRIVO_WRITE_LIMIT`, padrão 120; 0 desliga), contados só depois do CSRF, para que pedidos forjados não gastem a cota de ninguém. Atrás de proxy, com `CRIVO_TRUSTED_PROXY=true`, o IP é o último do `X-Forwarded-For`. Leituras não contam	429 com `Retry-After`
JSON	Texto UTF-8 válido, objeto na raiz e sem aninhamento abusivo (um JSON com milhares de níveis é recusado em vez de derrubar o processo)	422
Projeto	Todas as regras de "Validação antes de gravar" do capítulo 3, incluindo o RLE de cada célula	422
Miniatura	Data URL de imagem PNG cujos bytes começam com a assinatura `89 50 4E 47 0D 0A 1A 0A`, até 512 KB. Um arquivo de outro tipo renomeado para `.png` é recusado	422
Concorrência	`SELECT ... FOR UPDATE` e número de revisão (capítulo 6)	409

Revisões como rede de segurança

Toda gravação que passa pelas etapas acima guarda antes o estado anterior em `revision`. Uma alteração indevida, ou um salvamento por cima depois de um conflito, pode ser desfeita restaurando a versão anterior; o limite de `CRIVO_MAX_REVISIONS` versões por projeto evita que a tabela cresça sem controle.

Leituras que não alteram nada

As rotas de leitura (`/export`, `/png`, `/thumbnail`) só fazem `SELECT`. A rota `/png` compõe as camadas visíveis em memória e limita o lado final a 2048 px. As respostas JSON da API saem com `Cache-Control: no-store`.

> O Crivo não tem contas de usuário: proteja o endereço no proxy (autenticação, VPN ou rede interna), use HTTPS com `CRIVO_HTTPS=true` e uma `CRIVO_SECRET_KEY` própria.

Capítulo 8

Consultas prontas

Projetos mais recentes

```
SELECT p.name, p.width || 'x' || p.height AS size,
       (SELECT count(*) FROM frame f WHERE f.project_id = p.id) AS frames,
       (SELECT count(*) FROM layer l WHERE l.project_id = p.id) AS layers,
       p.revision, p.updated_at
FROM project p
ORDER BY p.updated_at DESC
LIMIT 20;
```

Espaço ocupado por projeto

```
SELECT p.name,
       coalesce(sum(c.size_bytes), 0) AS pixel_bytes,
       (SELECT coalesce(sum(r.size_bytes), 0) FROM revision r WHERE r.project_id = p.id) AS
revision_bytes,
       coalesce(length(p.thumbnail), 0) AS thumbnail_bytes
FROM project p
LEFT JOIN frame f ON f.project_id = p.id
LEFT JOIN cell c ON c.frame_id = f.id
GROUP BY p.id
ORDER BY pixel_bytes DESC;
```

Taxa de compressão das células

```
SELECT p.name, f.position + 1 AS frame, l.name AS layer,
       p.width * p.height * 4 AS raw_bytes,
       c.size_bytes,
       round(100.0 * c.size_bytes / (p.width * p.height * 4), 1) AS percent
FROM cell c
JOIN frame f ON f.id = c.frame_id
JOIN layer l ON l.id = c.layer_id
JOIN project p ON p.id = f.project_id
ORDER BY percent DESC;
```

Cores de uma paleta

```
SELECT pa.name, pc.position, pc.hex
FROM palette pa
JOIN palette_color pc ON pc.palette_id = pa.id
WHERE pa.slug = 'pico-8'
ORDER BY pc.position;
```

Versões de um projeto

```
SELECT r.number, r.name, r.size_bytes, r.created_at
FROM revision r
JOIN project p ON p.id = r.project_id
WHERE p.name = 'Poção'
ORDER BY r.number DESC;
```

Tags e modos de mistura

```
SELECT p.name AS project, t.position, t.name AS tag, t.direction, t.repeat,
       ff.position + 1 AS first_frame, ft.position + 1 AS last_frame
FROM tag t
JOIN project p ON p.id = t.project_id
JOIN frame ff ON ff.project_id = p.id AND ff.key = t.from_frame_key
JOIN frame ft ON ft.project_id = p.id AND ft.key = t.to_frame_key
ORDER BY p.name, t.position;

SELECT blend_mode, kind, count(*) AS layers
FROM layer
GROUP BY blend_mode, kind
ORDER BY layers DESC;
```

Projetos com miniatura suspeita

A API já recusa miniaturas que não são PNG, mas bancos antigos ou dados inseridos direto pelo SQL podem ter outra coisa. Esta consulta lista o que não começa com a assinatura do PNG:

```
SELECT p.id, p.name, length(p.thumbnail) AS bytes
FROM project p
WHERE p.thumbnail IS NOT NULL
  AND substring(p.thumbnail FROM 1 FOR 8) <> '\x89504e470d0a1a0a'::bytea;
```

Capítulo 9

Como preparar o banco

Com Docker (recomendado)

O `docker-compose.yml` já sobe o PostgreSQL 17 com o banco, o usuário e a senha do `.env`, num volume chamado `crivo_dados-postgres` (o mesmo da 1.x, para que os dados continuem lá). A aplicação espera o banco ficar saudável, roda `migrate` e começa a atender. Não há nada a fazer além de rodar `start.cmd` ou `./start.sh`; ao atualizar da 1.x, a migração `0003_english_schema` roda sozinha nessa hora, e ao atualizar da 2.0, a `0004_aseprite_features`.

PostgreSQL próprio

```
CREATE ROLE crivo WITH LOGIN PASSWORD 'troque-esta-senha';
CREATE DATABASE crivo OWNER crivo ENCODING 'UTF8';
```

```
export CRIVO_DB_ENGINE=postgresql
export CRIVO_DB_HOST=localhost CRIVO_DB_PORT=5432
export CRIVO_DB_NAME=crivo CRIVO_DB_USER=crivo CRIVO_DB_PASSWORD=troque-esta-senha
python manage.py migrate
```

Qualquer PostgreSQL a partir da versão 14 atende. Os gerenciados (Amazon RDS, Azure, Google Cloud SQL, Supabase, Neon) funcionam do mesmo jeito; se o serviço exigir SSL, acrescente `sslmode=require` nas opções de conexão em `crivo/settings.py`.

O usuário `crivo` precisa apenas ser dono do próprio banco. Não use o superusuário `postgres` na aplicação: se a senha vazar, o estrago fica restrito ao banco do Crivo.

Backup e restauração

```
docker compose exec postgres pg_dump -U crivo -d crivo -Fc > crivo.dump
docker compose exec -T postgres pg_restore -U crivo -d crivo --clean --if-exists < crivo.dump
```

O primeiro comando gera um backup compacto; o segundo restaura por cima do banco atual. Como os projetos também podem ser baixados um a um como `.crivo`, outra forma simples de backup é guardar esses arquivos. Faça um backup antes de atualizar da 1.x para a 2.0.0 ou da 2.0.0 para a 2.1.0.

Desempenho

- As listas de projetos usam o índice `project_updated_idx` e não leem a coluna `thumbnail`.
- Abrir um projeto lê as células com uma única consulta por projeto.
- Células e versões são `bytea`. Valores grandes vão automaticamente para o armazenamento TOAST do PostgreSQL, comprimidos, sem pesar nas varreduras das outras colunas.
- Gravações usam `bulk_create` em lotes de 200 células, dentro de uma transação.

Capítulo 10

Evolução do esquema

O esquema é criado e alterado só por migrações do Django, na pasta `editor/migrations`:

Migração	O que faz
`0001_inicial`	Cria as sete tabelas, as chaves, os índices e as restrições (com os nomes em português da 1.x)
`0002_paletas_predefinidas`	Cria as paletas Crivo, Game Boy, NES, Pico-8, CGA, CGA modo 4, Commodore 64 e 1 bit, com uma cópia fixa dos dados de então (a lista atual fica em `editor/palettes.py`)
`0003_english_schema`	Renomeia modelos, tabelas, colunas, restrições e o índice para o inglês, sem copiar dados, e marca os projetos com `format_version = 2`
`0004_aseprite_features` NOVO	Acrescenta as colunas e as tabelas dos recursos do Aseprite, com as restrições, sem reescrever nenhuma linha

Versão do Crivo	Mudança no banco
1.0.0	Esquema inicial (migrações 0001 e 0002)
1.1.0	Nenhuma. Ferramentas novas no editor, rota `/png`, formatos C e Java no servidor e proteções da camada HTTP, todas sem tocar nas tabelas
1.2.0	Nenhuma. App desktop e arquivo único, que guardam os projetos no navegador
2.0.0	Nomes em inglês, pela migração `0003_english_schema`; os dados ficam onde estão
2.1.0	Recursos do Aseprite, pela migração `0004_aseprite_features`: colunas novas em `project`, `layer` e `cell` e as tabelas `tag`, `slice` e `tileset`

Migração para nomes em inglês

A `0003_english_schema` usa só operações de renomear (`RenameModel`, `AlterModelTable` e `RenameField`), então nenhuma linha é copiada e o tempo de migração não depende do tamanho do banco. Ela roda numa transação e faz, em ordem:

1. remove o índice e as restrições com nomes em português, enquanto ainda apontam para as colunas antigas;
2. renomeia os modelos e as tabelas e depois as colunas e chaves estrangeiras;
3. recria as restrições e o índice com os nomes em inglês do capítulo 3 (`project_width_valid`, `layer_key_unique`, `project_updated_idx`...), com as mesmas regras;
4. marca todo projeto com `format_version = 2`, porque a API passou a devolver documentos no formato 2;
5. no PostgreSQL, renomeia também os nomes gerados a partir das tabelas antigas (chaves primárias, checagens, índices das chaves estrangeiras e sequências), para que o esquema inteiro fique em inglês.

Tabela na 1.x	Tabela na 2.0.0	Colunas renomeadas
`projeto`	`project`	nome, largura, altura, versao_formato, revisao, paleta, miniatura, criado_em, atualizado_em → name, width, height, format_version, revision, palette, thumbnail, created_at, updated_at
`camada`	`layer`	projeto_id, chave, posicao, nome, visivel, opacidade, bloqueada → project_id, key, position, name, visible, opacity, locked
`quadro`	`frame`	projeto_id, chave, posicao, duracao_ms → project_id, key, position, duration_ms
`celula`	`cell`	quadro_id, camada_id, formato, tamanho_bytes → frame_id, layer_id, encoding, size_bytes
`revisao`	`revision`	projeto_id, numero, nome, conteudo, tamanho_bytes, criado_em → project_id, number, name, content, size_bytes, created_at
`paleta`	`palette`	nome, descricao, predefinida, criado_em, atualizado_em → name, description, preset, created_at, updated_at
`cor_paleta`	`palette_color`	paleta_id, posicao → palette_id, position

As versões guardadas em `revision.content` não são reescritas: as anteriores à 2.0.0 continuam no formato 1 e são convertidas quando restauradas (capítulo 6). Consultas e relatórios em SQL feitos para a 1.x precisam usar os nomes novos, como os do capítulo 8. A migração pode ser desfeita com `python manage.py migrate editor 0002`, que volta os nomes da 1.x; faça um backup antes de atualizar.

Migração dos recursos do Aseprite

A `0004_aseprite_features` só acrescenta. Toda coluna nova tem valor padrão ou aceita nulo, então os projetos existentes continuam lidos exatamente como antes, e a API devolve os documentos da 2.0 sem mudança. Ela faz, numa transação:

1. acrescenta `project.color_mode` (padrão `rgba`);
2. acrescenta em `layer`: `blend_mode` (padrão `normal`), `kind` (padrão `image`), `parent_key` (nulo), `collapsed` e `reference` (falsos) e `tileset_key` (nulo);
3. acrescenta em `cell`: `link_frame_key` e `tiles`, ambos nulos;
4. cria as tabelas `tag`, `slice` e `tileset`, cada uma com `project_id` (cascata), `key` e `position` únicas no projeto;
5. cria as restrições `CHECK` e `UNIQUE` do capítulo 3, no mesmo estilo da 0003.

Tabela	Colunas novas
`project`	`color_mode varchar(10) NOT NULL DEFAULT 'rgba'`
`layer`	`blend_mode varchar(16)`, `kind varchar(8)`, `parent_key varchar(40) NULL`, `collapsed boolean`, `tileset_key varchar(40) NULL`, `reference boolean`
`cell`	`link_frame_key varchar(40) NULL`, `tiles bytea NULL`

Para voltar, `python manage.py migrate editor 0003` remove as colunas e as tabelas novas (e com elas os grupos, modos, tags, slices e tilesets gravados); faça um backup antes.

Como mudar o esquema

Para mudar o modelo, altere `editor/models.py`, gere a migração e aplique:

```
python manage.py makemigrations editor
python manage.py migrate
python manage.py makemigrations --check --dry-run
```

O último comando é o mesmo que a integração contínua roda: ele falha se o código e as migrações estiverem fora de sincronia. Se a mudança alterar o formato do arquivo, aumente `FORMAT_VERSION` em `js/core/format.js` e `FORMAT_VERSION` em `editor/services.py`, e trate a versão antiga em `deserialize` e em `upgrade_document`, para que projetos já salvos continuem abrindo.